Тема 20–21: Софтуерна архитектура. Проектиране и документиране.

ТЕОРИЯ

1. Дефиниция на софтуерна архитектура

Софтуерна архитектура = съвкупност от **структури**, показващи различните софтуерни елементи на системата, техните **външно видими свойства** и **връзките** между тях.

Архитектурата е **абстракция** — разглежда елементите като **черни кутии** (не се интересуваме от алгоритми, а от взаимодействията).

Понятие	Значение
Структура	Съвкупност от елементи, видими свойства и връзки
Изглед (View)	Конкретно документирано представяне на дадена структура
Елемент	Модул, компонент, процес — третира се като черна кутия
Външно видимо свойство	Интерфейс на елемента (API, протокол)

---

2. Структури и изгледи

Три вида архитектурни структури

Структура	Описва	Пример
Модулна	Функционалните компоненти и взаимовръзките между тях (статично)	Модул за поръчки, плащания, потребители
На процесите	Динамичното поведение по време на изпълнение	Как заявка преминава през слоевете
На разполагането (Физическа)	Как компонентите са разпределени на хардуерни ресурси	Уеб сървър, БД сървър, кеш сървър

Четири вида архитектурни изгледи

Изглед	Описва	UML диаграми
Логически	Основните абстракции (класове, компоненти)	Class diagram, Component diagram
На процесите	Взаимодействащи процеси при изпълнение	Sequence diagram, Activity diagram
На кода	Организация на файловете и пакетите	Package diagram
Физически	Разпределение на компоненти по хардуерни възли	Deployment diagram

---

3. Нефункционални изисквания (Quality Attributes)

Качествата поставят **ограничения** върху начина на работа на системата.

Качество	Описание
Надеждност	Работи правилно при очаквани и неочаквани условия
Производителност	Отговор в разумно време при очакван товар
Скалируемост	Способност да расте с увеличаване на натоварването
Сигурност	Защита от неоторизиран достъп
Наличност (Availability)	% от времето системата е достъпна
Изменяемост	Лесно добавяне на нова функционалност
Тестваемост	Лесно тестване на компонентите
Преносимост	Работа на различни платформи
Използваемост	Лесна употреба от потребителите

Сценарии за качество (Quality Scenarios)

Инструмент за **формализиране** на нефункционалните изисквания. Всеки сценарий има **6 компонента**:

Компонент	Описание	Пример
Контекст	Кога/при какви условия	„По време на работа на системата"
Източник	Кой/какво предизвиква събитието	„Потребителят"
Въздействие	Какво действие се извършва	„Прави годишна справка за продажбите"
Обект	Върху какво е въздействието	„Системата"
Резултат	Какво се случва	„Обработва заявката и връща PDF"
Количествени параметри	Измерима мярка ← **НАЙ-ВАЖНО**	„За по-малко от 15 секунди"

---

4. Процес на проектиране

Архитектурни drivers = най-приоритетните функционални + нефункционални изисквания, трансформирани в:

- **Use-cases** (функционални)
- **Сценарии за качество** (нефункционални)

Процесът е цикличен: нови изисквания → промяна на архитектурата.

Последователност (ADD — Attribute-Driven Design):

```
1. Анализ на изисквания → функционални + нефункционални изисквания
2. Избор на архитектурни drivers (приоритизиране)
3. Декомпозиция на системата на модули
4. За всеки модул:
   а) Избор на drivers за модула
   б) Избор на тактики и шаблони
   в) Декомпозиция на подмодули
   г) Приписване на функционалност
   д) Диаграми (class, sequence, deployment…)
   е) Проверка дали отговаря на изисквания
   ж) Рекурсивно задълбаване при нужда
5. Определяне на връзки между модулите
6. Оценка, валидация и документиране
```

---

5. Архитектурни тактики

Конкретни решения за постигане на желани качествени характеристики:

Надеждност

- **Ping/Echo, Heartbeat, Keepalive** — компонент А пуска сигнал, очаква отговор в интервал
- **Активен излишък** — дублиране на компоненти; при отказ → незабавен превключване
- **Пасивен излишък** — резервен компонент стартира при отказ на основния

Производителност

- **Кеширане** — съхраняване на чести резултати
- **Load balancing** — разпределяне на натоварването
- **Паралелна обработка** — конкурентни заявки
- **По-мощен хардуер**

Сигурност

- **Криптиране** на данните (TLS, AES)
- **Хеширане** на пароли (bcrypt, SHA-256)
- **Двуфакторна автентикация**

Скалируемост

- **Хоризонтално скалиране (scale out)** — добавяне на повече машини
- **Вертикално скалиране (scale up)** — по-мощен хардуер
- **Sharding** — разделяне на базата данни

Изменяемост

- **Open-Closed Principle** — отворен за разширения, затворен за промени

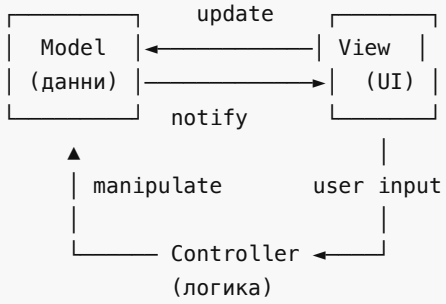
---

6. Архитектурни стилове

Стил	Описание	Приложение
Pipe and Filter	Данни → филтри (паралелно) → резултат; гъвкав, паралелизъм	Компилатори, ETL pipeline

Layered	Вертикални слоеве; всеки ползва слоя под него	ОС, TCP/IP, enterprise приложения
Client-Server	Сървър предлага ресурси; клиент консумира	Web приложения
Shared-Data	Централно хранилище (Repository/Blackboard)	СУБД-базирани системи
Implicit Invocation	Събитийна шина; компонентите си комуникират асинхронно	Event-driven systems
MVC	Model-View-Controller; разделяне на данни, UI, логика	Web frameworks
Microservices	Малки автономни услуги; независим деплоймент	Netflix, Amazon
SOA	Услуги с дефинирани интерфейси; SOAP/REST	Enterprise интеграция

MVC детайлно



- **Model**: данни + бизнес логика; независим от UI
- **View**: визуализация; зависи от Model
- **Controller**: обработва входа; координира Model и View

---

7. Документиране на архитектурата

Предназначение

- **Комуникация** между заинтересованите лица
- **Поддръжка и развитие** на системата
- Основа за разработката

Основни принципи

- Знай кой ще чете документацията
- Постави се на мястото на читателя
- Ясна, точна, добре структурирана

Съдържание

Секция	Съдържание
Общ преглед	Основните структури и компоненти
Архитектурни изгледи	Логически, на кода, на процесите, физически

Нефункционални изисквания	Критерии за качество
Описание на елементите	Детайли за всеки елемент и връзките му
Архитектурни решения	Обосновка на взетите решения
Вариации	Алтернативни подходи
Терминологичен речник	Дефиниции на термините

Структура на документацията

1. **Въведение** — цели, обхват, общ поглед
2. **Архитектурни изгледи** — подробни изгледи
3. **Сценарии за качество**
4. **Архитектурни решения** — обосновка
5. **Интеграция и тестване**
6. **Поддръжка и развитие**

КАК СЕ РЕШАВАТ ЗАДАЧИ

> ***Task 6*** *на практическия изпит (2021–2024): Модулна декомпозиция от изисквания R1–Rn + обосновка.*

Тип 1: Модулна декомпозиция по дадени изисквания

Задача: Дадени са изисквания R1–R8 за система за онлайн библиотека. Изградете модулна декомпозиция.

Метод (стъпки):

1. **Групирай изискванията** по функционална свързаност
2. **Дефинирай модулите** (всеки модул = група свързани изисквания)
3. **Опиши ролята** на всеки модул
4. **Покажи зависимостите** между модулите
5. **Обоснова избора** (кои изисквания → кой модул)

Пример:

Изисквания:

- R1: Потребителят може да търси книги по заглавие/автор
- R2: Потребителят може да резервира книга
- R3: Библиотекарят може да добавя/редактира книги
- R4: Потребителят трябва да влезе с акаунт
- R5: Системата изпраща известие при готова резервация
- R6: Администраторът управлява потребители
- R7: Системата пази история на заемите
- R8: Потребителят може да вижда профила си

Модулна декомпозиция:

```
Онлайн библиотека
├── Модул за управление на книги (R1, R3)
```

```
│   ├── Каталог (R1) – търсене по заглавие/автор
│   └── Администрация (R3) – добавяне/редактиране
├── Модул за резервации (R2, R5, R7)
│   ├── Управление на резервации (R2)
│   ├── Нотификации (R5) – известия
│   └── История (R7) – история на заемите
└── Модул за потребители (R4, R6, R8)
    ├── Автентикация (R4) – вход/регистрация
    ├── Администрация (R6) – управление на акаунти
    └── Профил (R8) – личен профил
```

Обосновка (задължително!):

> *„Модулът за управление на книги обединява R1 и R3, тъй като те се занимават с каталога от книги. Разделянето им в различни модули би довело до тясно свързване и дублиране на данни. Модулът за резервации обединява R2, R5 и R7, тъй като всички са свързани с жизнения цикъл на резервацията — от създаването до историята. Нотификациите (R5) са в него, тъй като се задействат при промяна на резервация. Модулът за потребители обединява R4, R6 и R8, тъй като управляват идентичността и профила."*

Тип 2: Построяване на друг изглед по дадена декомпозиция

Задача: По дадена модулна декомпозиция → изграждане на **изглед на процесите** (sequence diagram).

Пример: Проследяване на процеса "Потребителят резервира книга":

```
User      AuthModule    BookModule    ReservationModule    NotifModule
 |             |              |                |                |
 |--login()-->|              |                |                |
 |<--session--|              |                |                |
 |--search()-------------->  |                |                |
 |<--bookList--------------- |                |                |
 |--reserve(bookId)------------------------->|                |
 |                           |  checkAvail()->|                |
 |                           |  <--ok---------|                |
 |                           |                |--sendNotif()-->|
 |<--------------------------|--------ok------|                |
```

Тип 3: Обосновка на архитектурно решение

Задача: Системата трябва да поддържа 10 000 едновременни потребители (нефункционално изискване). Предложи архитектурна тактика.

Отговор (по структура):

1. **Качествен атрибут**: Производителност + Скалируемост
2. **Сценарий**: При пиково натоварване от 10K потребители, системата трябва да отговори за < 500ms
3. **Тактика**: Load balancing (хоризонтално скалиране) + Кеширане на чести заявки
4. **Обосновка**: Load balancer разпределя заявките между N инстанции на приложението; кешът намалява I/O към базата данни

Бърза справка

Понятие	Ключ
Архитектура =	Структури + видими свойства + връзки
Структура ≠ изглед	Структурата е реална; изгледът е документирана проекция
Модулна структура	Статична; функционални компоненти
Структура на процесите	Динамична; изпълнение
Физическа структура	Хардуерно разполагане
Сценарий за качество	6 компонента; количествените параметри са ключови
ADD	Attribute-Driven Design; цикличен процес
Pipe-and-Filter	Филтри + канали; паралелизъм
MVC	Model + View + Controller
Load balancing	Хоризонтална скалируемост
Heartbeat	Тактика за надеждност
Декомпозиция	Разбиване на система на модули по функция